

# Architettura AI a Bassa Latenza per l'Orchestrazione Zero Trust Data-Driven

## 1. Fondamenti del Paradigma Zero Trust e Sfide Architetturali

L'evoluzione inarrestabile delle infrastrutture informatiche moderne, profondamente caratterizzate dall'adozione massiccia di paradigmi legati al cloud computing, alle architetture a microservizi e all'accesso distribuito delle identità, ha reso fundamentalmente obsoleti i tradizionali modelli di sicurezza basati sul perimetro.<sup>1</sup> Il framework Zero Trust Architecture (ZTA) è emerso come risposta architetturale imperativa per mitigare queste vulnerabilità. Il fulcro concettuale del paradigma Zero Trust risiede nell'abbandono dell'assunto di fiducia implicita legata alla topologia di rete, per imporre invece un meccanismo di difesa proattivo basato sulla verifica continua e sulla valutazione dinamica dell'affidabilità.<sup>2</sup> I principi cardine di tale architettura impongono che nessuna entità – sia essa interna o esterna al perimetro aziendale – debba essere considerata intrinsecamente sicura ("never trust by default"), che ogni singola richiesta di accesso debba essere sottoposta a un'autenticazione multidimensionale ("always verify continuously") e che i diritti di accesso debbano essere limitati allo stretto indispensabile per l'esecuzione del task ("enforce least privilege").<sup>1</sup>

Tuttavia, l'implementazione di una ZTA puramente deterministica, basata esclusivamente su insiemi di regole statiche o su firme di attacco predefinite, risulta profondamente inadeguata nell'identificare minacce emergenti o in continua evoluzione.<sup>3</sup> All'interno dei log di Identity and Access Management (IAM), un comportamento anomalo può apparire statisticamente innocuo se analizzato in isolamento, ma rivelarsi contestualmente malevolo se inserito in una sequenza più ampia, segnalando attacchi sofisticati come l'escalation dei privilegi, il movimento laterale o le minacce interne (insider threats).<sup>3</sup> Di conseguenza, la ricerca accademica e industriale ha orientato i propri sforzi verso l'integrazione di sistemi di User and Entity Behavior Analytics (UEBA), trasformando le ZTA da set di policy statiche in meccanismi di valutazione continua della fiducia guidati dai dati (data-driven continuous trust evaluation).<sup>1</sup>

Il principale ostacolo ingegneristico per la realizzazione di tali sistemi risiede nella necessità di garantire una valutazione del rischio in tempo reale e un'autorizzazione adattiva in scenari caratterizzati da alta concorrenza e requisiti di bassissima latenza.<sup>1</sup> Nel contesto del rilevamento delle anomalie di rete, la tempestività è un fattore critico: le anomalie possono manifestarsi sotto forma di picchi improvvisi di volume, alterazioni nelle sequenze di richieste o connessioni da intervalli IP estranei.<sup>4</sup> Affinché l'infrastruttura di sicurezza, governata da un orchestratore come Open Policy Agent (OPA), possa intervenire preventivamente prima che il danno venga consumato, il modello di Intelligenza Artificiale deputato al calcolo dello "score di anomalia" deve necessariamente produrre un'inferenza entro una finestra temporale rigorosa. I requisiti industriali stabiliscono che tale processo debba avvenire in un arco di tempo idealmente

inferiore ai 10 ms, o comunque con un limite massimo tollerabile di 40 ms.<sup>1</sup> Ritardi superiori, misurati in frazioni di secondo o secondi, invalidano l'efficacia della mitigazione automatizzata (ad esempio contro attacchi DDoS a insorgenza rapida) e degradano inaccettabilmente la User Experience dei servizi aziendali.<sup>4</sup>

Il presente documento fornisce un'analisi architetturale esaustiva per lo sviluppo di un modello AI che rispetti pedissequamente tali vincoli, esaminando la rappresentazione persistente dello stato, la topologia ottimale delle reti neurali, le tecniche avanzate di compilazione del grafo computazionale, e l'integrazione a bassa latenza all'interno di un ecosistema nativo sviluppato in linguaggio Go.

## **2. Rappresentazione dei Dati e Gestione dello Stato: Dai Log ai Grafi**

Il requisito fondamentale per valutare la "legalità" di una richiesta utente in base alla sua storia pregressa, al dispositivo utilizzato e alla rete di provenienza, impone una rigorosa riprogettazione della modalità con cui i log transazionali vengono archiviati, indicizzati e successivamente forniti al modello di intelligenza artificiale per l'inferenza.<sup>2</sup>

### **2.1 Criticità delle Architetture Documentali (MongoDB)**

L'ipotesi di raccogliere i log provenienti dal security orchestrator, fonderli con i metadati delle richieste e immagazzinarli in un database documentale come MongoDB presenta limitazioni strutturali intrinseche e fatali per i requisiti di latenza imposti dal progetto. MongoDB, pur offrendo un'eccellente scalabilità orizzontale e flessibilità schematica basata su formati BSON, è progettato intorno all'indicizzazione documentale e all'attraversamento di strutture ad albero B-Tree.

Per estrarre il "sotto-grafo" storico di un utente e ricostruire la complessa rete di relazioni tra l'identità, le sessioni passate, i dispositivi condivisi e le risorse a cui si è acceduto, sarebbe necessario ricorrere all'operatore di aggregazione \$lookup.<sup>6</sup> Quando applicato su larga scala per emulare attraversamenti multi-hop (ad esempio: trovare l'utente, trovare tutti i dispositivi usati dall'utente, trovare tutti gli altri utenti che hanno usato quegli stessi dispositivi, e verificare le loro attività), l'operatore \$lookup innesca colli di bottiglia severi a livello di elaborazione CPU e I/O su disco.<sup>6</sup> Operazioni di aggregazione relazionale su milioni di record documentali richiedono inevitabilmente decine o centinaia di millisecondi, saturando istantaneamente il budget di latenza totale stabilito (10-40 ms) ancor prima che il dato raggiunga il modello neurale. Ottenere risposte in frazioni di millisecondo per estrarre uno storico relazionale complesso è meccanicamente in conflitto con il paradigma architetturale di MongoDB.

### **2.2 Transizione Architetturale verso i Database a Grafo In-Memory**

Per superare i limiti relazionali dei database documentali, i log IAM, le tracce di audit e gli eventi di trasmissione dati devono essere modellati fin dalla loro genesi come grafi dinamici eterogenei.<sup>3</sup> In questa topologia, i nodi rappresentano entità distinte (utenti, ruoli, sessioni, dispositivi fisici, subnet IP, risorse di destinazione), mentre gli archi direzionali rappresentano

interazioni temporali e azioni di accesso.<sup>3</sup> Questa astrazione matematica consente ai sistemi informatici di catturare le dipendenze temporali, relazionali e contestuali che altrimenti sfuggirebbero a un'analisi tabulare piatta.<sup>2</sup>

Tuttavia, all'interno dell'ecosistema dei Graph Database, sussistono variazioni prestazionali di estrema magnitudo. Soluzioni tradizionali basate su persistenza a disco, come Neo4j, sebbene potenti per l'analisi offline, faticano a garantire latenze sub-millisecondo. Benchmark industriali evidenziano come, su carichi di lavoro complessi basati su query Cypher, database tradizionali possano esibire latenze reali che fluttuano tra i 13,7 ms e i 3,1 secondi.<sup>7</sup> Considerando che l'inferenza AI consumerà la maggior parte dei 10-40 ms disponibili, l'estrazione dei dati (data retrieval) non può eccedere i 2-3 ms.

L'adozione di database a grafo *in-memory* sviluppati specificamente per l'altissima velocità è quindi una necessità imprescindibile. Soluzioni come Memgraph o FalkorDB (precedentemente noto come RedisGraph) operano direttamente all'interno della Random Access Memory (RAM), bypassando completamente la latenza associata all'I/O dei dispositivi di archiviazione non volatili. Memgraph, ad esempio, è capace di ingerire 100.000 nodi in soli 400 ms e di risolvere query strutturali complesse con latenze operative nell'ordine degli 1,09 ms.<sup>7</sup> Analogamente, l'integrazione di FalkorDB in architetture ad alta intensità ha dimostrato la capacità di eseguire attraversamenti a 7 hop abbassando la latenza media a 0,3 ms, garantendo la fornitura fulminea del sotto-grafo richiesto.<sup>8</sup>

| Caratteristica Architetture       | MongoDB (Aggregazioni \$lookup)   | Database a Grafo su Disco (es. Neo4j) | Database a Grafo In-Memory (es. Memgraph / FalkorDB) |
|-----------------------------------|-----------------------------------|---------------------------------------|------------------------------------------------------|
| Paradigma di Rappresentazione     | Documentale (BSON)                | Grafo Nativo (LPG)                    | Grafo Nativo in RAM                                  |
| Ottimizzazione delle Relazioni    | Emulata (costosa via join/lookup) | Eccellente (Puntatori diretti)        | Superiore (Puntatori in memoria)                     |
| Latenza di Attraversamento Tipica | Decine/Centinaia di millisecondi  | 13 ms - 3.1 s <sup>7</sup>            | < 1.1 ms <sup>7</sup>                                |
| Compatibilità con SLA 10-40 ms    | Totalmente Inadeguato             | Rischioso (Varianza elevata)          | Altamente Raccomandato                               |

La progettazione ottimale prevede che l'orchestratore Go instradi i log in tempo reale verso Memgraph/FalkorDB. Al momento della richiesta di accesso, l'orchestratore interroga il database in memoria estraendo unicamente l'"Ego-Network" dell'utente (ovvero il nodo utente e i suoi vicini fino al k-esimo grado di separazione), ottenendo in meno di un millisecondo il contesto strutturale necessario alla valutazione AI.

### 3. Analisi e Selezione dell'Architettura del Modello Neurale

Il requisito progettuale riguardante la "generalizzazione" impone un vincolo ferreo sulla tipologia di apprendimento automatico da adottare. Il sistema deve essere in grado di operare efficacemente essendo addestrato raramente (solo in caso di ricalibrazione delle policy di base) e deve rilevare autonomamente comportamenti imprevisti o variazioni nei pattern di accesso senza necessitare di etichette per ogni possibile e futura tipologia di attacco. Questo indirizza inequivocabilmente la selezione architetturale verso algoritmi di Anomaly Detection operanti in regime non supervisionato o semi-supervisionato.<sup>4</sup>

### **3.1 Critica alle Architetture Proposte: Transformer, CNN e Modelli Statistici**

L'idea di avvalersi di architetture standard come Transformer o Reti Convoluzionali (CNN) per l'analisi della storia dell'utente e del suo sotto-grafo presenta incompatibilità algoritmiche significative rispetto alle latenze richieste e alla natura dei dati.

Le Reti Convoluzionali (CNN), anche in combinazione con moduli ricorrenti (CNN-LSTM) capaci di latenze sotto i 50 ms in contesti specifici<sup>5</sup>, possiedono un bias induttivo rigido che richiede dati spazialmente strutturati in griglie euclidee (come i pixel di un'immagine o le matrici NCHW tipiche del formato tensoriale visivo).<sup>9</sup> Convertire l'intrinseca topologia non lineare e asimmetrica di un sotto-grafo relazionale in una matrice densa comporta la distruzione delle informazioni di connettività strutturale e genera enormi tensori sparsi, causando colli di bottiglia nel calcolo e il fenomeno dell'over-smoothing, rendendo i nodi indistinguibili tra loro.<sup>2</sup>

Allo stesso modo, i modelli basati su architettura Transformer, sebbene rivoluzionari nel catturare dipendenze a lungo raggio tramite il meccanismo della Self-Attention, soffrono di una

complessità computazionale di ordine quadratico  $O(N^2)$  rispetto alla lunghezza della sequenza in ingresso. Passare un'intera storia di log utente a un Transformer costringerebbe il sistema a calcolare i pesi di attenzione per ogni coppia possibile di eventi. Senza meccanismi di sparsificazione avanzata, questo processo introduce una latenza intrinseca difficilmente comprimibile al di sotto della soglia dei 10 ms su singole inferenze online (batch size = 1), oltre a non modellare nativamente le intersezioni tra domini (es. IP e Device che confluiscono sullo stesso utente) come farebbe un grafo.

Infine, le tecniche di Anomaly Detection statistico-tradizionali, come le Isolation Forest, godono di ampia notorietà per la loro scalabilità e velocità, riuscendo a processare set di dati dell'ordine dei 100 milioni di record con tassi di utilizzo CPU prossimi al 100% in addestramento e latenze di scoring minime.<sup>10</sup> Ciononostante, il loro funzionamento si fonda sulla creazione di partizioni decisionali allineate agli assi (axis-aligned splits) operanti su feature tabulari mutualmente indipendenti.<sup>11</sup> Questo impedisce categoricamente alle Isolation Forest di comprendere i percorsi causali multi-hop e le relazioni strutturali tipiche dell'Identity and Access Management, rendendole inadatte alla valutazione Zero Trust che si fonda sulla correlazione continua delle entità.<sup>3</sup>

### **3.2 Graph Neural Networks (GNN) come Soluzione Architetturale Ottimale**

Le Reti Neurali su Grafo (Graph Neural Networks o GNN) rappresentano l'unico framework matematico in grado di fondere l'intelligenza relazionale con le rigorose procedure del paradigma Zero Trust.<sup>2</sup> Le GNN operano direttamente sulle topologie a grafo, astruendo entità complesse e tracciando l'evoluzione temporale dei comportamenti senza imporre strutture a griglia innaturali.<sup>2</sup> L'inclusione delle GNN nell'architettura Zero Trust dota il sistema della capacità di "ragionamento a livello semantico", estendendo l'analisi dalla singola domanda "chi è l'utente?" alla valutazione olistica "questo utente si sta comportando secondo i canoni operativi storici stabiliti dall'identità della sua macchina, della sua posizione e delle sue autorizzazioni?".<sup>2</sup> Il nucleo computazionale di una GNN si fonda su un meccanismo di aggregazione iterativa definito Message Passing. Ad ogni layer neurale, un nodo aggiorna il proprio vettore di caratteristiche (embedding) aggregando matematicamente i vettori provenienti dai nodi adiacenti.<sup>12</sup> Questo processo è formalizzato dalla seguente equazione:

$$\mathbf{h}_v^{(l)} = \text{UPD} \left( \text{AGG} \left( \left\{ \mathbf{h}_u^{(l-1)} * \text{MSG}(\mathbf{h}_u^{(l-1)}, \mathbf{h}_v^{(l-1)}) \mid u \in \mathcal{N}(v) \right\} \right), \mathbf{h}_v^{(l-1)} \right)$$

In questa formulazione,  $\mathbf{h}_v^{(l)}$  rappresenta la rappresentazione vettoriale latente (l'embedding) del nodo  $v$  al livello  $l$ , la funzione MSG elabora il messaggio lungo gli archi connessi,

l'operatore AGG sintetizza i segnali provenienti dal vicinato  $\mathcal{N}(v)$ , e la funzione UPD aggiorna lo stato del nodo unendo le informazioni storiche con i nuovi input raccolti.<sup>12</sup> Iterando questo processo per un numero ristretto di layer (tipicamente 2 o 3), il modello cattura dipendenze strutturali di alto ordine.<sup>2</sup>

Per ottemperare ai requisiti di generalizzazione senza ricorrere a etichettatura manuale delle anomalie, l'architettura implementerà un **Graph Autoencoder (GAE)**. Durante la fase di addestramento off-line, il GAE è istruito esclusivamente su log benigni e consolidati, imparando a comprimere la topologia relazionale in uno spazio latente e, successivamente, a ricostruire gli archi e le feature del grafo originale. La funzione di ottimizzazione cerca di minimizzare l'errore tra il grafo in ingresso e il grafo ricostruito.

In fase di inferenza (a runtime), quando un utente compromesso tenta un'operazione non in linea con i pattern appresi, il decodificatore del GAE fallirà nel ricostruire le nuove interazioni devianti. Questo fallimento si traduce in un valore quantitativo definito "Reconstruction Loss" o "Reconstruction Error". Tale errore matematico è proporzionale all'anomalia comportamentale del sotto-grafo.<sup>3</sup> Normalizzando il valore della Loss in un intervallo probabilistico \$\$, si ottiene lo "score di legalità" richiesto, che può essere trasmesso ad OPA per determinare il blocco o l'assenso della richiesta. L'utilizzo di robusti meccanismi di message passing, unitamente all'applicazione sistematica della tecnica *DropEdge* durante il training, preserverà il modello dall'over-fitting e assicurerà un'elevata generalizzazione contro le dinamiche avversariali emergenti.<sup>2</sup>

## 4. Strategie di Inferenza a Bassissima Latenza su Grafi

# Evolving

La sfida computazionale più gravosa legata all'adozione delle GNN risiede nel loro impiego su grafi in continua evoluzione (streaming graphs).<sup>12</sup> Negli utilizzi tradizionali, le GNN sono elaborate su grafi statici tramite un processo di elaborazione in blocco (batch computation) offline, che richiede tempo per propagare le modifiche sull'intera struttura reticolare.<sup>12</sup> Operare un aggiornamento completo del grafo aziendale ad ogni singola richiesta di accesso genererebbe colli di bottiglia macroscopici, impossibilitando categoricamente il raggiungimento di tempi di reazione inferiori a 40 ms.<sup>12</sup>

La soluzione all'avanguardia per bypassare tale limitazione è l'adozione del paradigma **Runtime Embedding Computation (RTEC)** per GNN incrementali.<sup>12</sup> RTEC scinde nettamente la gestione della storia profonda dalla necessità di valutazione online.

L'infrastruttura RTEC per l'Anomaly Detection Zero Trust si articolerà nelle seguenti fasi operative:

1. **Aggiornamento Background (Offline/Asincrono):** Mentre le richieste transitano a milioni nell'ambiente aziendale, un processo distaccato elabora iterativamente le informazioni accumulate nel database in memoria (es. Memgraph), calcolando e immagazzinando gli embedding di base per ogni utente, risorsa e dispositivo. Questo allevia il sistema di sicurezza in linea dal dover compiere pesanti operazioni matematiche sull'intero reticolo.<sup>12</sup>
2. **Calcolo Incrementale dell'Ego-Network (Online/Sincrono):** Nel momento in cui giunge una singola interrogazione ad OPA, il database estrae unicamente l'Ego-Network locale (i nodi immediatamente contigui) dell'entità richiedente. Questi pochi nodi, uniti ai loro embedding asincroni già calcolati in precedenza, vengono inviati alla rete neurale ONNX in un tensore estremamente ridotto. La GNN a runtime deve unicamente elaborare un calcolo vicinato-specifico ("neighbor-wise computation") circoscritto ai vertici interessati dalla transazione corrente.<sup>12</sup>

Questa implementazione incrementale elimina i calcoli ridondanti, abbattendo dal 64% al 99% il volume di elaborazione richiesto per l'inferenza, con incrementi di velocità dimostrati rispetto alle GNN ingenua che vanno da un fattore di 1.7x fino a 145.8x.<sup>12</sup> La prova inconfutabile che le reti neurali su grafo possano soddisfare e superare gli SLA di 10 ms giunge dai recenti studi nel campo della fisica delle alte energie, dove modelli GNN dinamici schierati su acceleratori hardware hanno dimostrato la capacità di processare collisioni particellari raggiungendo un throughput di 2,94 milioni di eventi al secondo e latenze end-to-end clamorosamente basse pari a 7,15 microsecondi (0,00715 ms), ampiamente inferiori ai vincoli dell'infrastruttura Zero Trust.<sup>15</sup>

## 5. Esportazione del Grafo Computazionale e Compilazione TensorRT

Il modello GNN, progettato e validato nel solido ecosistema di ricerca Python (PyTorch e

PyTorch Geometric), deve essere inevitabilmente convertito in un formato diagnostico indipendente, capace di operare nativamente in ambienti di produzione C++ o Golang, scisso dalle pesanti inefficienze dell'interprete Python (Global Interpreter Lock, cicli dinamici, gestione dinamica della memoria).<sup>16</sup> Il formato designato come standard de facto per l'interoperabilità ad alte prestazioni è **ONNX (Open Neural Network Exchange)**.<sup>16</sup>

## 5.1 Il Motore AOT PyTorch Dynamo

Nelle incarnazioni storiche di PyTorch, l'esportazione si basava sul tracciatore TorchScript, il quale presentava limitazioni nella cattura del flusso di controllo. Le pipeline moderne richiedono l'utilizzo del motore avanzato basato su `torch.export`, attivabile tramite il flag `dynamo=True` nella funzione `torch.onnx.export`.<sup>17</sup> L'esportatore Dynamo agisce in ottica Ahead-Of-Time (AOT), estraendo un grafo di pura logica tensoriale e trasformando ogni operazione complessa in istruzioni del set operativo normalizzato ATen.<sup>18</sup> Qualsiasi diramazione Python viene eliminata a priori e tradotta in operatori condizionali strettamente pre-calcolabili.<sup>19</sup> Per massimizzare le performance di recupero, parametri di pesatura strutturali pre-addestrati vengono fissati permanentemente nel grafo (`InputKind.PARAMETER`), garantendo che il motore in esecuzione possa applicare tecniche rigorose di fusione dei layer (`layer fusion`), accorpando ad esempio convoluzioni su grafi e funzioni di attivazione non lineari.<sup>18</sup>

## 5.2 L'Insidia delle "Dynamic Shapes" e L'Ottimizzazione del Profilo

La criticità assoluta nell'esportazione di modelli che elaborano Ego-Network risiede nella variabilità intrinseca del dato in ingresso: ogni richiesta interrogherà un sotto-grafo di dimensioni differenti a seconda del contesto relazionale dell'utente. A prima vista, l'impiego del parametro `dynamic_shapes` (o `dynamic_axes`) durante la chiamata a `torch.onnx.export` sembrerebbe la prassi corretta per indicare al compilatore l'uso di vincoli di range (es. la definizione simbolica `s0: VR[2, int_oo]`, che segnala una dimensione variabile ma strutturalmente superiore a 2).<sup>18</sup>

Eppure, a livello operativo profondo, l'abilitazione incondizionata dei tensori a forma dinamica all'interno dei motori di esecuzione ad altissime prestazioni come NVIDIA TensorRT introduce overhead temporali talvolta letali per l'obiettivo dei 10 ms.<sup>21</sup> La documentazione architetturale Nvidia rivela che per gestire forme non prefissate, TensorRT adotta una strategia esecutiva a "ping-pong" tra CPU e GPU <sup>21</sup>:

1. Il motore delega alla CPU il calcolo procedurale dei metadati (`Shape Tensor`), determinando le dimensioni fisiche dell'operazione.
2. I dati esecutivi (`Execution Tensor`) vengono trasferiti sulla GPU per l'elaborazione del layer AI.
3. Qualora un layer debba ricalcolare la forma in uscita in base a parametri non noti in fase di build (ad esempio riduzioni dimensionali o operazioni topologiche complesse su sotto-grafi), si impone una rigorosa barriera di sincronizzazione. Il tensore di forma viene forzatamente riversato dalla VRAM (GPU) alla memoria di sistema della CPU. Questo "context switch" bidirezionale paralizza le code di elaborazione CUDA e innesca latenze incontrollabili.<sup>23</sup>

A peggiorare il quadro diagnostico, quando TensorRT incontra un sotto-grafo con un numero di nodi mai processato in precedenza, fallisce nel trovare riscontri nella sua "Layer-level Cache" o nella "Kernel-level Cache" e delega il problema alla "Selection-level Cache".<sup>21</sup> Ciò causa un tempo di stallo severo in cui il compilatore Just-In-Time seleziona e istanzia nuovi kernel specifici per le dimensioni ignote, provocando pesanti anomalie nel throughput che possono durare diverse decine di millisecondi.<sup>21</sup> Benché un approccio basato sulla calibrazione formale dei profili ottimali (Optimization Profiles impostando min, opt, max shapes) argini questo deterioramento<sup>21</sup>, la presenza del passaggio da Shape Tensor a Execution Tensor permane.<sup>23</sup>

**Soluzione Architetture: Static Bucketing e Padding** Per sradicare preventivamente l'overhead descritto, è mandatorio ripudiare le esecuzioni puramente dinamiche a favore del "Padding Statico" accoppiato al "Bucketing". Verranno definiti a priori tre profili dimensionali strettamente statici, calcolati in fase costruttiva. Ad esempio, il Bucketing può prevedere un target piccolo per sotto-grafi con  $N = 32$  nodi, uno medio per  $N = 128$ , e uno grande per  $N = 512$ . Durante la richiesta in tempo reale, il codice di pipeline Golang estrarrà i vicini dell'utente dal DB In-Memory e applicherà un padding con tensori vuoti ("dummy nodes" esclusi tramite masking) per arrotondare il vettore al Bucket superiore più prossimo. Ciò consentirà a TensorRT di operare su forme assolutamente costanti, pre-allocando tutta la memoria VRAM in anticipo e attuando le più estreme tattiche di fusione del Kernel, garantendo inferenze stabili in meno di pochi millisecondi.<sup>22</sup>

| Modalità di Rappresentazione Vettoriale          | Vantaggi                                                                               | Svantaggi                                                                                             | Esito per Latenza <10ms                  |
|--------------------------------------------------|----------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|------------------------------------------|
| <b>Dynamic Shapes</b><br>(Dimensioni Variabili)  | Assenza di codice di riempimento (Padding) lato CPU                                    | Overhead critico TensorRT, strategia di scambio Ping-Pong CPU/GPU, ricalcolo JIT kernel <sup>21</sup> | Fallimento probabile per latenze estreme |
| <b>Static Shapes + Bucketing</b> (Profili Fissi) | Esecuzione deterministica, fusione ottimale, nessun ricalcolo in itinere <sup>22</sup> | Richiede pre-elaborazione logica (masking/padding) nel codice di embedding                            | Ottimale e Altamente Raccomandato        |

Inoltre, il modello salvato su disco o passato via flusso binario dovrà adottare un layout matriciale NCHW (numero di lotti, canali di caratteristica, dimensioni). I tentativi di utilizzare layout NHWC – comuni su TensorFlow/CPU – causerebbero la creazione occulta di operatori Transpose da parte del convertitore, sperperando microsecondi in copie memorieologiche superflue all'interno del motore di calcolo.<sup>9</sup>

## 6. Sviluppo, Testing e Allocazione dell'Hardware



Le risorse hardware a disposizione del progetto consentono la definizione di una pipeline di continuous integration e deployment ottimale, sfruttando i punti di forza delle architetture eterogenee AMD/NVIDIA e Apple Silicon.

## 6.1 L'Ambiente di Addestramento e Compilazione (Workstation AMD/NVIDIA)

Il sistema desktop equipaggiato con processore AMD Ryzen 9 9900X, 32 GB di RAM DDR5 ECC e la scheda grafica NVIDIA RTX 5070 Ti (16 GB) si profila come il server dedicato all'addestramento massivo (Offline GNN Embedding Generation), all'esportazione del modello e alla profilazione avanzata.

- **Capacità Computazionale VRAM:** I 16 GB di memoria video della RTX 5070 Ti garantiscono un buffer inestimabile per l'ingestione dell'intero dataset dei log storici necessari per ottimizzare il Graph Autoencoder e ricavare l'intricata geometria dei pesi, sfruttando librerie accelerate da CUDA (come PyTorch Geometric) che possono distribuire i lotti (batch) operando massivamente in parallelo.<sup>22</sup>
- **Precisione INT8/FP16:** Per incrementare vertiginosamente la reattività del file finale ONNX, la GPU NVIDIA può essere impiegata per effettuare quantizzazione statica (riducendo la precisione dei pesi da 32 bit Float a INT8 o FP16). Per l'impiego in INT8, il motore effettua calibrazioni minuziose impiegando uno dei profili costanti statici (Buckets), miscelando una frazione ridotta del dataset di validazione per determinare il clipping range dei layer senza degradare irrimediabilmente l'accuratezza diagnostica del ricostruttore topologico.<sup>23</sup>
- **Memoria ECC come Baluardo di Integrità:** Nell'ottica in cui questo nodo serva non solo da training set ma da gestore del processo in background RTEC per il Graph Database (es. Memgraph), la presenza della correzione di errore automatica (ECC RAM) assume importanza strategica. Eventuali bit-flip causati da radiazioni cosmiche in ambienti con altissimi tassi di transazioni in RAM causerebbero corruzioni latenti nelle matrici relazionali, inducendo a falsi positivi sistemici nella fase di inferenza del punteggio di sicurezza.<sup>12</sup>
- **Profilazione Critica:** Su questa postazione, utilizzando l'eseguibile di test `onnxruntime_perf_test.exe`, verrà attivato momentaneamente il livello di profilazione massima (JSON tracing mode o Nsight Systems via nsys) per dissezionare le performance nanosecondo per nanosecondo. Una menzione d'allerta severa proviene dalla diagnostica empirica: l'opzione `--nvtxMode=verbose` di TensorRT è afflitta da un grave difetto di stallo legato alla generazione delle stringhe C per i marker log, e deve essere categoricamente bandita e commutata in `--nvtxMode=none` durante la simulazione della produzione per preservare misurazioni attendibili sotto i 10 ms.<sup>24</sup>

## 6.2 L'Ambiente di Validazione dell'Orchestratore (MacBook Pro M4 Pro)

Il portatile basato su Apple Silicon M4 Pro (unbinned) con 24 GB di Unified Memory offre un substrato diametralmente opposto ma essenziale per il collaudo dell'orchestrazione software in

Go. A differenza delle architetture x86 convenzionali limitate dalle interconnessioni del bus PCI-Express, il paradigma Unified Memory Memory architecture (UMA) del processore M4 Pro permette al codice di elaborazione CPU scritto in Golang e all'Execution Provider di ONNX (che può sfruttare CoreML o la GPU integrata Apple) di condividere lo stesso pool di dati a banda larghissima senza replicare frammenti di tensori in spazi di memoria discreti. Questo simula le performance di altissimo livello tipiche di implementazioni Zero-Copy nei datacenter aziendali moderni, garantendo che l'interfaccia C-Go e le goroutine non inducano latenze transizionali durante il calcolo dell'Ego-Network del grafo.<sup>25</sup>

## 7. Esecuzione Nello Stack Golang via ONNX Runtime

Per onorare il requisito stringente relativo alla reattività intrinseca del Security Orchestrator (già implementato in linguaggio Go per evitare pause di garbage collection prolungate e ottimizzare la concorrenza), occorre valicare il muro architetturale che separa le infrastrutture ML storicamente relegate a C++ e Python, e il server aziendale Go.

La soluzione ottima prevede l'inclusione del wrapper [github.com/yalue/onnxruntime\\_go](https://github.com/yalue/onnxruntime_go).<sup>25</sup> Tra le librerie esaminate<sup>25</sup>, questo pacchetto brilla per una gestione profondamente robusta dell'infrastruttura di caricamento, superando le idiosincrasie associate all'ecosistema CGo (es. conflitti con Mingw e compilatori cruzati su piattaforme multiple). Piuttosto che forzare una compilazione statica del monumentale codice sorgente di Microsoft ONNX Runtime, l'architettura della libreria yalue attua un caricamento dinamico della libreria binaria pre-compilata ufficiale (libonnxruntime.so.1.x per server Linux, onnxruntime.dll per Windows, o libonnxruntime.dylib sul MacBook M4 Pro) all'avvio dell'applicazione tramite il comando esplicito `ort.SetSharedLibraryPath()`.<sup>25</sup> Questo consente di aggiornare indipendentemente l'infrastruttura di inferenza ML e l'applicativo binario dell'orchestratore Go senza temere regressioni a livello di linking.<sup>25</sup>

### 7.1 Configurazione della Sessione a Bassa Latenza (SessionOptions)

Una mera invocazione dei pesi in ONNX non assicura automaticamente latenze in frazioni di millisecondo. Il ciclo di vita della sessione deve essere curato con un'ingegneria dei flag estrema:

1. **Livelli di Ottimizzazione Topologica:** Il costruttore `NewORTSessionOptions` (o le sue equivalenti chiamate in ambiente yalue) sarà configurato tramite l'inserimento formale del livello di ottimizzazione `ORT_ENABLE_ALL`.<sup>9</sup> Come delineato dalla diagnostica sulle performance operative, affinché il compilatore interno ONNX proceda a eseguire tecniche capitali quali il *Constant Folding* (la pre-valutazione dei nodi che producono valori invariabili nel tempo) e l'*Operator Fusion* (l'accorpamento di complessi pattern matematici `MatMul+Add`), è imperativo che l'esportatore abbia marcato tutti i tensori strutturali come costanti scisse dagli input di grafo.<sup>9</sup>
2. **Mitigazione della Varianza via Dynamic Block Base:** I test empirici condotti su piattaforme multicore dotate di elevati numeri logici (caratteristica primaria sia del Ryzen 9 9900X che dell'M4 Pro) rivelano picchi anomali di latenza (latency variance) se il thread pool interno tenta di assegnare frammenti costanti (constant cost model) di task

inferenziali alle CPU in modo statico. Quando un nucleo operativo ultima la propria frazione di calcolo precocemente, lo stallo sincronico che ne consegue annienta il tempo di esecuzione dell'intero lotto.<sup>9</sup> ONNX Runtime espone un antidoto potente e configurabile associando l'ingresso diagnostico `session.dynamic_block_base = 4` al costruttore.<sup>9</sup> Con questo approccio, la ripartizione dei task adotta una curva di granularità decrescente obbedendo all'equazione del residuo

$$N / (\text{dynamic\_block} \times \text{numero\_thread})$$
, forzando le core CPU a prelevare quantità di elaborazione sempre più affinate verso la coda di completamento, azzerando virtualmente i picchi negativi di varianza ed esprimendo inferenze omogenee strettamente inscritte nel dominio temporale dei 10 ms.<sup>9</sup>

3. **Spinlock Non-Bloccanti:** Per minimizzare i cicli operativi bruciati dalle contestazioni dei Mutex (le attese di mutua esclusione sui thread), ONNX offre la facoltà di essere costruito a partire dai sorgenti abilitando code di compiti lock-free (`--use_spinlock`), che mantengono i thread vigili e reattivi rispetto agli ingressi di rete improvvisi.

## 8. Integrazione Architetture in Open Policy Agent (OPA)

Ottenuto il punteggio probabilistico (l'errore di ricostruzione del Graph Autoencoder) in scala \$\$ da ONNX Runtime Go, esso deve dirigere il motore decisionale asincrono del Security Orchestrator, il cui cervello logico è basato su Open Policy Agent (OPA).

### 8.1 Evitare l'Antipattern della Ricompilazione OPA

L'integrazione di sistemi decisionali avanzati in orchestrazioni Zero Trust precipita frequentemente in un collo di bottiglia critico dovuto al fraintendimento della dualità funzionale di OPA. OPA compie due tipologie diametralmente opposte di calcolo: la **Compilazione della Policy**, che consuma ingenti risorse CPU e I/O per convertire il codice dichiarativo in piani operativi, e la **Valutazione della Policy**, progettata per essere sbrigata in pochi microsecondi.<sup>30</sup> Se il sistema scritto in Golang invocasse ingenuamente un parsing delle istruzioni Rego relative alla sicurezza della ZTA per ogni singola transazione HTTP o grpc intercettata, si innescherebbe un loop disastroso in cui l'AST (Abstract Syntax Tree) verrebbe eretto da capo ad ogni istante. L'unico modello di integrazione ammesso in scenari operativi ad alto throughput richiede il disaccoppiamento totale del ciclo di vita della Policy rispetto al ciclo di vita della Request.<sup>30</sup> All'inizializzazione del demone in Go (fase di startup), l'eseguibile caricherà le policy globali in memoria, interpellando l'istruzione `PrepareForEval(context.Background())`.<sup>30</sup> L'esecuzione produrrà una struttura immutabile e ottimizzata definita `rego.PreparedEvalQuery`.<sup>30</sup> Successivamente, ad ogni milionesimo di tentativo di accesso, l'orchestratore non dovrà far altro che iniettare le dipendenze contestuali (IP sorgente, Timestamp, Endpoint desiderato) nell'handler compilato invocando il metodo `Eval(...)`. Questa procedura evita qualsiasi ritardo interpretativo, lasciando virtualmente intatto il budget dei 10-40 ms a completa e unica disposizione del tracciato ONNX e delle query In-Memory.

## 8.2 Passaggio del Punteggio tramite Custom Built-in Functions

La problematica terminale concerne il come immettere la valutazione di Intelligenza Artificiale all'interno del flusso dichiarativo di OPA, affinché gli ingegneri preposti alla sicurezza possano stilare policy chiare ed efficaci. Molte organizzazioni errano appoggiandosi a chiamate RESTful asincrone (tramite web-hook) che da sole implicano l'accumulo di ritardi TCP intollerabili. Esplorazioni di basso livello mostrano come un'eventuale manipolazione dello storage core di OPA (tramite moduli sperimentali per backend custom) sia ardua da far accettare all'upstream e fragile contro futuri aggiornamenti.<sup>31</sup>

Il paradigma eccellente, patrocinato dalla documentazione ufficiale per l'integrazione a libreria ("embedding OPA as a library" <sup>32</sup>), impiega lo strumento delle **Custom Built-in Functions** del linguaggio Go.<sup>32</sup> Questo meccanismo permette allo sviluppatore di immettere nuove funzionalità lessicali native all'interno del linguaggio interpretato Rego.

Durante l'inizializzazione dell'ambiente Golang globale, si provvede a dichiarare e registrare un puntatore di struttura &Builtin{} tramite il package astratto topdown.RegisterBuiltinFunc().<sup>31</sup>

Questa procedura stabilisce una ferma definizione dei tipi per i parametri d'ingresso in arrivo dalla richiesta OPA (es. stringhe types.Named("user\_info", types.S)) e una singola via d'uscita (il numero float della loss types.N).<sup>33</sup> La callback funzionale associata a questa Built-In non farà altro che eseguire una chiamata sincrona, prelevando le variabili di interesse passate dallo script di sicurezza, compiere l'estrazione veloce dal database a grafo in-memory (FalkorDB o Memgraph), elaborare il sub-grafo con la sessione ONNX Runtime in background precedentemente scaldato in RAM, e restituire in tempi minimali il valore matematico generato dalla GNN, fornendo il giudizio senza mai uscire dallo spazio logico unificato del processo. Conseguentemente, a livello delle policy operative dichiarative, il file .rego che governerà le transazioni finali apparirà semanticamente limpido e conciso, rendendo la sofisticazione neurale GNN un'esperienza in tempo reale integrata alla perfezione:

Snippet di codice

```
package authz.zero_trust
```

```
default allow = false
```

```
allow {
```

```
  # Verifica preventiva che i vincoli formali d'accesso e le rotte HTTP siano validati
```

```
  input.request.method == "POST"
```

```
  input.request.path == "/api/infrastruttura_protetta"
```

```
  # Invocazione della logica GNN Custom Built-In compilata via C-Go (Latenza < 10ms)
```

```
  anomaly_score := zta_compute_gnn_anomaly(input.user.id, input.device.fingerprint)
```

```
# Se il Graph Autoencoder non ritiene il pattern anomalo (score sotto la soglia), consente.  
anomaly_score < 0.15  
}
```

In sintesi, la sinergia progettuale che intreccia un database a grafo in memoria capace di alimentare tensori d'ingresso istantanei (Ego-Network extraction), un modello ad alte prestazioni basato su Autoencoder per dati grafici (Graph Neural Network RTEC per la generalizzazione intrinseca di vettori di fiducia continui) esente da costrutti dinamici paralizzanti in fase TensorRT, unitamente alla spina dorsale C-Go orchestrata dai compilatori PrepareForEval di OPA e funzioni Custom Built-in per il passaggio del segnale in RAM, assicura la realizzazione di un Security Orchestrator Zero Trust radicalmente adattivo, robusto dinanzi ai flussi intensivi di rete, e inflessibile nella protezione dei vincoli temporali di latenza estrema richiesti dalle operazioni moderne.

## Bibliografia

1. Research on data transmission anomaly detection based on zero-trust architecture and graph neural networks - AIP Publishing, accesso eseguito il giorno maggio 19, 2026, <https://pubs.aip.org/aip/adv/article/16/1/015222/3378065/Research-on-data-transmission-anomaly-detection>
2. (PDF) Research on data transmission anomaly detection based on zero-trust architecture and graph neural networks - ResearchGate, accesso eseguito il giorno maggio 19, 2026, [https://www.researchgate.net/publication/400081680\\_Research\\_on\\_data\\_transmission\\_anomaly\\_detection\\_based\\_on\\_zero-trust\\_architecture\\_and\\_graph\\_neural\\_networks](https://www.researchgate.net/publication/400081680_Research_on_data_transmission_anomaly_detection_based_on_zero-trust_architecture_and_graph_neural_networks)
3. Graph Neural Network-Based Adaptive Threat Detection for Cloud Identity and Access Management Logs - arXiv, accesso eseguito il giorno maggio 19, 2026, <https://arxiv.org/html/2512.10280v1>
4. Network Anomaly Detection: A Comprehensive Guide - Kentik, accesso eseguito il giorno maggio 19, 2026, <https://www.kentik.com/kentipedia/network-anomaly-detection/>
5. Design of an improved graph-based model for real-time anomaly detection in healthcare using hybrid CNN-LSTM and federated learning - PMC, accesso eseguito il giorno maggio 19, 2026, <https://pmc.ncbi.nlm.nih.gov/articles/PMC11696656/>
6. Understanding MongoDB \$lookup performance - DEV Community, accesso eseguito il giorno maggio 19, 2026, <https://dev.to/lusqua/understanding-mongodb-lookup-performance-1115>
7. Memgraph vs Neo4j in 2025: Real-Time Speed or Battle-Tested Ecosystem? - Medium, accesso eseguito il giorno maggio 19, 2026, <https://medium.com/decoded-by-datacast/memgraph-vs-neo4j-in-2025-real-time-speed-or-battle-tested-ecosystem-66b4c34b117d>
8. How Securin Runs 7-Hop Threat Intelligence Queries in Under 350ms with

- FalkorDB, accesso eseguito il giorno maggio 19, 2026,  
<https://www.falkordb.com/case-studies/securin-falkordb-graph-case-study/>
9. Troubleshooting performance issues - onnxruntime, accesso eseguito il giorno maggio 19, 2026,  
<https://onnxruntime.ai/docs/performance/tune-performance/troubleshooting.html>
  10. Isolation forest final stage very slow and single threaded · Issue #13295 - GitHub, accesso eseguito il giorno maggio 19, 2026,  
<https://github.com/scikit-learn/scikit-learn/issues/13295>
  11. [2603.20335] Hybrid Autoencoder-Isolation Forest approach for time series anomaly detection in C70XP cyclotron operation data at ARRONAX - arXiv, accesso eseguito il giorno maggio 19, 2026, <https://arxiv.org/abs/2603.20335>
  12. Incremental GNN Embedding Computation on Streaming Graphs - arXiv, accesso eseguito il giorno maggio 19, 2026, <https://arxiv.org/html/2603.20622v1>
  13. Build a GNN-based real-time fraud detection solution using Amazon SageMaker, Amazon Neptune, and the Deep Graph Library | Artificial Intelligence, accesso eseguito il giorno maggio 19, 2026,  
<https://aws.amazon.com/blogs/machine-learning/build-a-gnn-based-real-time-fraud-detection-solution-using-amazon-sagemaker-amazon-neptune-and-the-deep-graph-library/>
  14. Embedding generation - Anyscale, accesso eseguito il giorno maggio 19, 2026,  
<https://www.anyscale.com/ai-embeddings>
  15. Reconfigurable Computing Challenge: Real-Time Graph Neural Networks for Online Event Selection in Big Science - arXiv, accesso eseguito il giorno maggio 19, 2026, <https://arxiv.org/html/2605.10612v1>
  16. Make Your Models Fly: ONNX Runtime Latency Gains for Pytorch | by Manevaish - Medium, accesso eseguito il giorno maggio 19, 2026,  
<https://medium.com/@manevaish17/make-your-models-fly-onnx-runtime-latency-gains-for-pytorch-with-9e26d645889e>
  17. Export a PyTorch model to ONNX — PyTorch Tutorials 2.12.0+cu130 documentation, accesso eseguito il giorno maggio 19, 2026,  
[https://docs.pytorch.org/tutorials/beginner/onnx/export\\_simple\\_model\\_to\\_onnx\\_tutorial.html](https://docs.pytorch.org/tutorials/beginner/onnx/export_simple_model_to_onnx_tutorial.html)
  18. torch.onnx — PyTorch 2.12 documentation, accesso eseguito il giorno maggio 19, 2026, <https://docs.pytorch.org/docs/stable/onnx.html>
  19. torch.export-based ONNX Exporter — PyTorch 2.12 documentation, accesso eseguito il giorno maggio 19, 2026,  
[https://docs.pytorch.org/docs/stable/onnx\\_export.html](https://docs.pytorch.org/docs/stable/onnx_export.html)
  20. How to get dynamic batch size in Onnx model from Pytorch? - Stack Overflow, accesso eseguito il giorno maggio 19, 2026,  
<https://stackoverflow.com/questions/76980330/how-to-get-dynamic-batch-size-in-onnx-model-from-pytorch>
  21. Working with Dynamic Shapes — NVIDIA TensorRT for RTX, accesso eseguito il giorno maggio 19, 2026,  
<https://docs.nvidia.com/deeplearning/tensorrt-rtx/latest/inference-library/work-wi>

[th-dynamic-shapes.html](#)

22. What is the difference between dynamic and static shape support in TensorRT?, accesso eseguito il giorno maggio 19, 2026, <https://massedcompute.com/faq-answers/?question=What%20is%20the%20difference%20between%20dynamic%20and%20static%20shape%20support%20in%20TensorRT?>
23. Dynamic Shapes: Advanced Topics — NVIDIA TensorRT, accesso eseguito il giorno maggio 19, 2026, <https://docs.nvidia.com/deeplearning/tensorrt/latest/inference-library/dynamic-shapes-advanced.html>
24. dynamic shape run supper slow and static shape run normal · Issue #3231 · NVIDIA/TensorRT - GitHub, accesso eseguito il giorno maggio 19, 2026, <https://github.com/NVIDIA/TensorRT/issues/3231>
25. value/onnxruntime\_go: A Go (golang) library wrapping ... - GitHub, accesso eseguito il giorno maggio 19, 2026, [https://github.com/value/onnxruntime\\_go](https://github.com/value/onnxruntime_go)
26. value/onnxruntime\_go\_examples: Example applications using the onnxruntime\_go library. - GitHub, accesso eseguito il giorno maggio 19, 2026, [https://github.com/value/onnxruntime\\_go\\_examples](https://github.com/value/onnxruntime_go_examples)
27. onnxruntime\_go package - github.com/value/onnxruntime\_go - Go Packages, accesso eseguito il giorno maggio 19, 2026, [https://pkg.go.dev/github.com/value/onnxruntime\\_go](https://pkg.go.dev/github.com/value/onnxruntime_go)
28. onnxruntime package - github.com/ivansuteja96/go-onnxruntime - Go Packages, accesso eseguito il giorno maggio 19, 2026, <https://pkg.go.dev/github.com/ivansuteja96/go-onnxruntime>
29. onnxruntime\_go package - github.com/value/onnxruntime - Go Packages, accesso eseguito il giorno maggio 19, 2026, <https://pkg.go.dev/github.com/value/onnxruntime>
30. Compile Once, Evaluate Many: Using OPA Correctly in Production | by Sushan Sapaliga, accesso eseguito il giorno maggio 19, 2026, <https://medium.com/@sushansapaliga/compile-once-evaluate-many-using-opa-correctly-in-production-039bea4ddf25>
31. runtime: add custom storage backend registration API · Issue #8277 · open-policy-agent/opa, accesso eseguito il giorno maggio 19, 2026, <https://github.com/open-policy-agent/opa/issues/8277>
32. Extending OPA - Open Policy Agent, accesso eseguito il giorno maggio 19, 2026, <https://openpolicyagent.org/docs/extensions>
33. Adding Built-in Functions - Open Policy Agent, accesso eseguito il giorno maggio 19, 2026, <https://openpolicyagent.org/docs/contrib-adding-builtin-functions>